

PIAZZA DEI MESTIERI – TORINO

IeFP – Operatore Informatico | Prima Annualità (Classe B22-19-20270)

PROGRAMMAZIONE 1

Fondamenti di Logica, Algoritmi e Sviluppo Software

Manuale Didattico Ufficiale del Corso di Laboratorio (80 ore)

Docente: Daniele Cerrina

Email: daniele.cerrina@immaginazioneelavoro.it

Anno Formativo: 2026 / 2027

Programmazione 1 – Fondamenti e Laboratorio

Percorso triennale IeFP per la Qualifica di Operatore Informatico.

© 2026–2027 Piazza dei Mestieri / Immaginazione e Lavoro.

Docente: Daniele Cerrina.

Documento didattico in continuo aggiornamento durante l'anno formativo.

Compilato con $\text{\LaTeX}$ e $\text{\pdflatex}$.

Presentazione del Libro

Questo volume rappresenta il punto di riferimento e il libro di testo per il corso di **Programmazione 1** (80 ore di laboratorio) presso la Piazza dei Mestieri di Torino.

L'approccio adottato è fortemente pratico e incrementale: ogni lezione svolta in aula e in laboratorio viene consolidata e integrata all'interno di questo testo, fornendo agli allievi:

- Le spiegazioni teoriche chiare dei concetti chiave.
- Gli esempi concreti e analogie vicine all'esperienza quotidiana e videoludica.
- Gli schemi logici ed esercizi svolti e da svolgere.
- Il percorso di sviluppo del **gioco guida annuale** votato dalla classe.

Come consultare questo documento: Il documento è provvisto di un **indice ipertestuale e cliccabile**. Cliccando su qualsiasi capitolo o sezione nell'Indice Generale o nei segnalibri del lettore PDF, verrai indirizzato direttamente alla pagina corrispondente.

Indice Generale

Presentazione del Libro	ii
I Pensiero Computazionale e Algoritmi	1
1 Introduzione agli Algoritmi e Pensiero Computazionale	2
1.1 Che cos'è un Algoritmo?	2
1.1.1 Esempi dal Quotidiano e dal Mondo Digitale	3
1.2 Le Quattro Proprietà Fondamentali	3
1.3 Esercizio Guidato: Attraversamento Pedonale	4
1.4 Verso le Prossime Lezioni	5
Guida Rapida alla Sintassi	6

Part I

Pensiero Computazionale e Algoritmi

Introduzione agli Algoritmi e Pensiero Computazionale

“Un algoritmo deve essere visto per essere creduto, ma soprattutto per essere compreso: è il ponte invisibile che trasforma un problema disordinato in una soluzione esatta.”

Riferimento Lezione: 21 Settembre 2026 – Prima Lezione di Laboratorio

Competenze chiave: Comprensione del modello Input/Output, identificazione delle proprietà di un algoritmo, formulazione di istruzioni non ambigue.

1.1 Che cos'è un Algoritmo?

Nel linguaggio comune siamo abituati a risolvere problemi ogni giorno senza pensare in modo analitico alle singole azioni che compiamo. Tuttavia, quando vogliamo delegare la risoluzione di un problema a un calcolatore (o spiegare a un'altra persona come agire senza margini d'errore), abbiamo bisogno di un metodo rigoroso.

Definizione di Algoritmo

Un **algoritmo** è una sequenza finita e ordinata di istruzioni elementari, chiare e non ambigue, la cui esecuzione porta alla risoluzione di un determinato problema in un tempo finito.

In informatica ogni problema viene formalizzato attraverso il modello fondamentale di elaborazione:

- **Input (Dati di ingresso):** Rappresenta l'informazione o la condizione iniziale fornita all'algoritmo. È ciò che “entra” nel processo.
- **Elaborazione:** L'insieme ordinato di istruzioni elementari applicate ai dati di ingresso.
- **Output (Risultato):** Il dato finale o l'azione risultante al termine dell'elaborazione. È ciò che “esce” dal processo.

INPUT (Dati / Problema) $\longrightarrow$ [**ALGORITMO**]
 $\longrightarrow$ **OUTPUT** (Risultato / Soluzione)

1.1.1 Esempi dal Quotidiano e dal Mondo Digitale

Gli algoritmi non appartengono solo al codice sorgente: li sperimentiamo quotidianamente in molteplici contesti:

Algoritmi nella vita reale

1. **Tutorial per costruire una casa in Minecraft:** Un elenco ordinato di azioni (1. Raccogli 64 blocchi di legno, 2. Costruisci una base 6×6 , 3. Innalza i muri per 4 blocchi, 4. Posiziona la porta e le torce). Se inverti i passaggi (ad esempio posizionare il tetto prima dei muri), l'obiettivo fallisce.
2. **Ricetta di cucina (es. Torta al cioccolato):** L'elenco degli ingredienti rappresenta l'**Input**; la sequenza dei passaggi (pesare, mescolare, infornare a 180 gradi per 35 minuti) costituisce l'**Algoritmo**; il dolce pronto è l'**Output**.
3. **Istruzioni di montaggio (LEGO o mobili IKEA):** Diagrammi e numeri sequenziali guidano la costruzione partendo dai singoli pezzi fino all'oggetto completo.

1.2 Le Quattro Proprietà Fondamentali

Non ogni elenco di frasi o istruzioni può essere definito un algoritmo valido. Per essere considerato formalmente **corretto**, un algoritmo deve soddisfare quattro caratteristiche essenziali:

1. **Finitezza:** L'algoritmo deve necessariamente terminare dopo aver eseguito un numero finito di istruzioni. Un procedimento che gira all'infinito senza mai giungere a una conclusione non è un algoritmo.
2. **Definitezza (Non ambiguità):** Ogni istruzione deve essere univoca e comprensibile senza alcuna interpretazione soggettiva. Frasi come "mescola un po'" o "aspetta qualche secondo" sono inaccettabili in un algoritmo informatico: bisogna specificare "mescola per 3 minuti a velocità 2" oppure "attendi 5 secondi".
3. **Efficienza:** L'algoritmo deve risolvere il compito impiegando una quantità ragionevole di tempo e di memoria.

Nota Didattica per la Prima Annualità

Nel corso di quest'anno la priorità assoluta sarà comprendere la **logica** e raggiungere la **correttezza**; la misurazione scientifica dell'efficienza (complessità computazionale) sarà affrontata negli anni successivi.

4. **Generalità:** L'algoritmo non deve risolvere un singolo caso specifico e isolato, ma deve essere in grado di risolvere **qualsiasi problema appartenente a quella specifica classe**.

Il Principio della Generalità

Il nostro obiettivo annuale è imparare a risolvere qualsiasi problema della categoria che stiamo affrontando.

Esempio pratico: Se progettiamo un algoritmo per calcolare la media aritmetica di 7 numeri, l'algoritmo deve funzionare correttamente sia con la sequenza $[1, 2, 3, 4, 5, 6, 7]$, sia con $[9, 8, 7, 6, 5, 4, 3]$, sia con qualsiasi altra combinazione di 7 valori numerici, inclusi numeri decimali o negativi.

1.3 Esercizio Guidato: Attraversamento Pedonale

Mettiamoci alla prova formulando un algoritmo esatto per un problema che affrontiamo ogni giorno.

Attraversare la strada in presenza di un semaforo

Scrivere l'algoritmo passo-passo, il più rigoroso e aderente possibile alle quattro proprietà esaminate, per risolvere il seguente problema:

Problema: Mi trovo sul marciapiede davanti alle strisce pedonali e devo attraversare la strada in presenza di un semaforo pedonale. Cosa faccio?

Algoritmo con Scelta Condizionale e Ripetizione

Ecco la formalizzazione dell'algoritmo divisa in istruzioni elementari:

1. **Osservazione:** Guardo il semaforo pedonale.
2. **Controllo condizionale:**
 - **SE** il semaforo è **VERDE**:
 - (a) Inizio ad attraversare a passo costante sulle strisce pedonali.
 - (b) Faccio un passo dopo l'altro **FINCHÉ** non ho raggiunto il marciapiede opposto.
 - (c) Fine dell'algoritmo (Obiettivo raggiunto con successo!).
 - **ALTRIMENTI** (il semaforo è rosso o giallo):
 - (a) Resto fermo sul marciapiede di partenza in sicurezza.
 - (b) Aspetto che il semaforo cambi stato.
 - (c) **Torno al punto 1** (ripeto l'osservazione).

Attenzione all'ambiguità e ai cicli infiniti!

- Se dimentichiamo la condizione "**FINCHÉ** non arrivo dall'altra parte", una persona programmata come un computer continuerebbe a camminare dritto all'infinito anche dopo aver superato la strada!
- Se omettiamo di dire cosa fare nel caso il semaforo sia rosso, il calcolatore si bloccherebbe per mancanza di istruzioni.

1.4 Verso le Prossime Lezioni

Nelle prossime lezioni trasformeremo questo modo di pensare in due linguaggi formali utilizzati universalmente dai programmatori di tutto il mondo:

- I **Diagrammi di Flusso (Flowchart)**: La rappresentazione grafica degli algoritmi tramite blocchi geometrici (rettangoli per le azioni, rombi per le condizioni, frecce per la direzione).
- Lo **Pseudocodice**: Un linguaggio a metà strada tra la lingua naturale e i linguaggi di programmazione reali (come Python e C), che ci permetterà di scrivere logiche complesse prima di passare alla tastiera del computer.

Guida Rapida alla Sintassi

In questa appendice finale verranno raccolti durante l'anno i promemoria di sintassi, le parole chiave e i comandi più utilizzati in laboratorio.